Inteligência Artificial

Redes Perceptron de camada simples

Prof. Dr. Jefferson Beethoven Martins

IFTM Campus UPT – 23/02/2026

https://g1.globo.com/jornal-nacional/noticia/2026/02/19/brasil-franca-e-india-se-unem-em-defesa-da-regulacao-mundial-da-inteligencia-artificial.ghtml

JORNAL NACIONAL

BUSCAR

Brasil, França e Índia se unem em defesa da regulação mundial da inteligência artificial

Entre os países reunidos na cúpula, se desenhou um quase consenso nos discursos: de que a IA precisa ser regulada de alguma forma. As gigantes de tecnologia também participaram.

Por Jornal Nacional

19/02/2026 21h15 · Atualizado há 3 dias

https://www.gov.br/secom/pt-br/acompanhe-a-secom/noticias/2026/02/r-4-7-bilhoes-para-projetos-de-inteligencia-artificial-ja-foram-aprovados-pelo-bndes

INSTITUTO FEDERAL
Triângulo Mineiro

Secretaria de Comunicação Social

O que você procura?

FINANCIAMENTO

R$ 4,7 bilhões para projetos de inteligência artificial já foram aprovados pelo BNDES

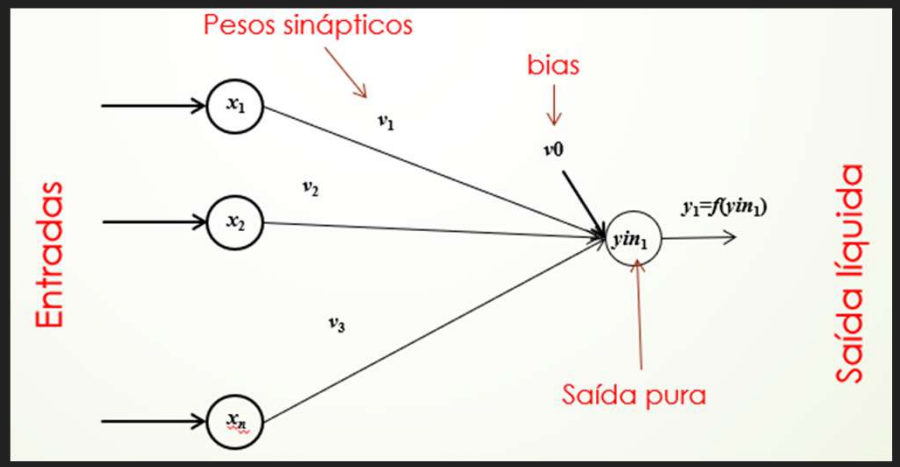
Recursos permitem investimentos em sistemas integradores e desenvolvedores, hardware e infraestrutura

Publicado em 20/02/2026 14h19

Compartilhe:

Rede Perceptron - Arquitetura

- Foi idealizado por Frank Rosenblatt em 1957.
- É a forma mais simples de configuração de uma RNA.
- Uma das aplicações iniciais consistia em identificar padrões geométricos.



Conceitos e nomeclatura

- As nomenclaturas dos pesos podem sofrer alterações dependendo do bibliografia da área. É possível encontrar a letra w ao invés da letra v para os pesos.
- Alguns autores costumam utilizar a letra b ou θ para o bias.
- Pertence a arquitetura feedforward (o fluxo de informações vai da camada de entrada para a camada de saída não havendo realimentação).

Parâmetros

Parâmetro	Representação	Tipo característico
Entradas	x_i	Reais ou binárias (pode ser necessário que sejam normalizados dependendo do problema
Pesos sinápticos	v_i	Reais (aleatórios entre -0,5 e 0,5)
Bias	$v0$	Reais (aleatórios entre -0,5 e 0,5)
Saída pura	yin	Real – Resultado da adição entre o bias e a somatória dos produtos entre as entradas e seus respectivos pesos.
Função de ativação	$f(yin)$	Degrau ou Degrau Bipolar
Saída Líquida	y	Binária ou Bipolar
Processo de treinamento	---------	Supervisionado

Do ponto de vista matemático

- Em termos matemáticos, a Rede Perceptron de camada simples consiste numa combinação linear, cuja formulação é mostrada abaixo:

$$y = \begin{cases} 1, & se \ \sum v_i \cdot x_i + v_0 \geq \text{limiar} \\ 0, & se \ \sum v_i \cdot x_i + v_0 < \text{limiar} \end{cases}$$

- ou

$$y = \begin{cases} 1, & se \ \sum v_i \cdot x_i + v_0 \geq \text{limiar} \\ -1, & se \ \sum v_i \cdot x_i + v_0 < \text{limiar} \end{cases}$$

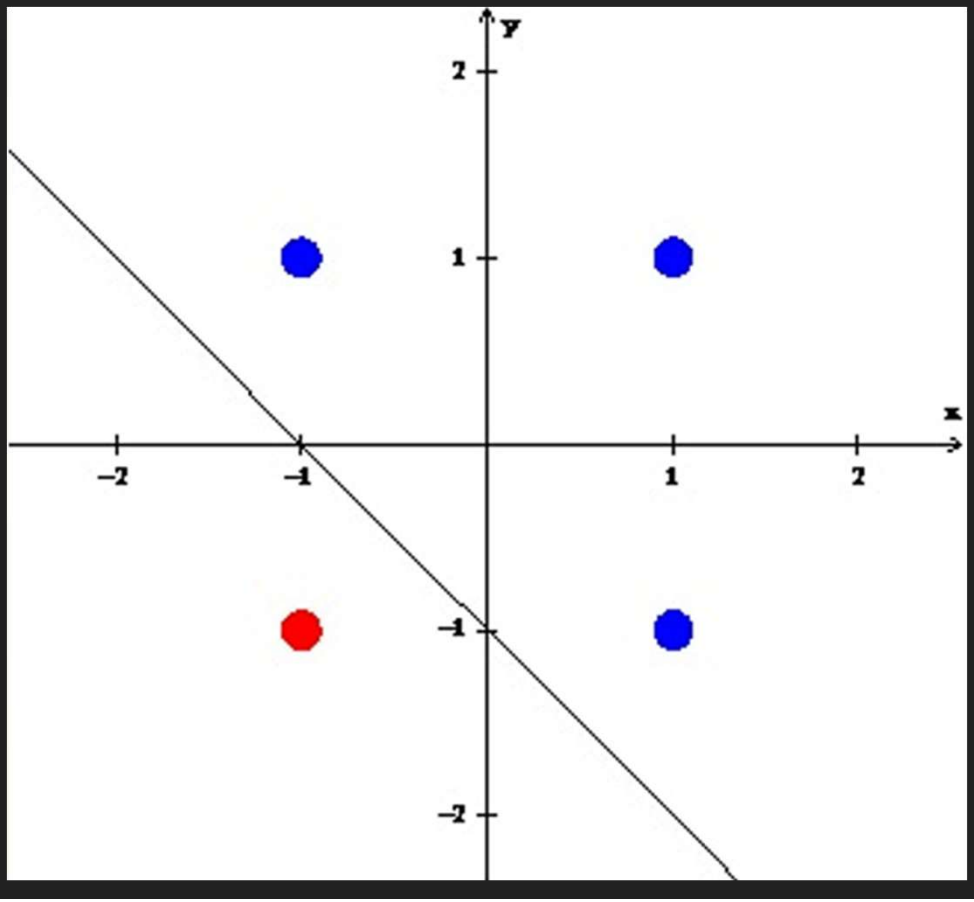
Do ponto de vista matemático

- Se a rede possuir 2 entradas, a expressão matemática será equivalente a:

$$v_1 \cdot x_1 + v_2 \cdot x_2 + v_0 = 0$$

- Nesse caso temos uma reta no plano cartesiano exercendo o papel de separador linear no plano cartesiano.
- Se a rede possuir 3 entradas, haverá um plano exercendo o papel de separador linear no espaço IR^3.
- Se a rede possuir n (n > 3) entradas, haverá um hiperplano exercendo o papel de separador linear no espaço IR^n.

Do ponto de vista matemático



Treinamento

- Treinar a rede consiste em encontrar os pesos sinápticos e bias de tal forma que a submissão das entradas conduza a saída desejada (target) para aquela amostra.
- Assim, o treinamento consiste em submeter as amostras de treinamento à RNA, comparar a saída obtida com a saída desejada e ajustar os pesos em caso de erro. Segue abaixo a formulação de atualização dos pesos:

Se $y \neq t$

$$w_i(novo) = w_i(velho) + \alpha t x_i$$

$$b(novo) = b(velho) + \alpha t$$

senão

$$w_i(novo) = w_i(velho)$$

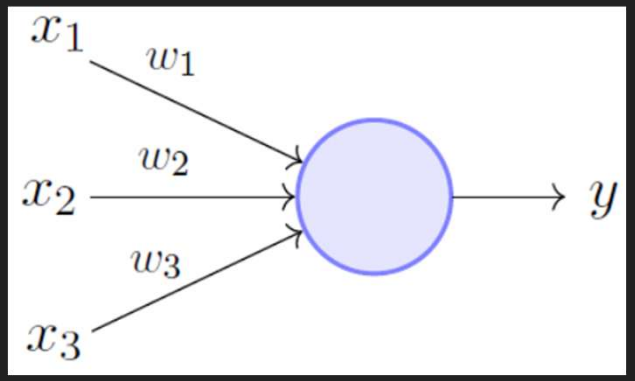
$$b(novo) = b(velho)$$

Treinamento

- O parâmetro α representa a "taxa de aprendizagem". Também é conhecido como "passo" da aprendizagem. Outros símbolos podem ser utilizados para essa representação como por exemplo a letra do alfabeto grego η.
- O valor de α indica o quão rápido deve ser o treinamento. A taxa de aprendizagem deve estar limitada ao intervalo [0,1]. Por mais tentador que possa ser um treinamento rápido, um valor muito alto pode causar instabilidade no treinamento e gerar um efeito reverso.
- Não há um método determinístico para o valor de α. Este valor depende do problema em que se está trabalhando, e da experiência de quem está modelando a RNA.

Treinamento

- Ao longo do treinamento, a rede vai ajustando a reta (plano ou hiperplano), até que o separador linear, seja capaz de separar os dois conjuntos de amostras de treinamento (supervisionado).
- O surgimento do erro e o método de atualização dos pesos foi uma grande novidade.



Detalhes

- O perceptron foi criticado por Marvin Minsky e Seymour Papert devido às suas limitações geométricas.
- Apenas problemas linearmente separáveis são resolvidos.
- Esse momento, na década de 60, foi conhecido como "O inverno da inteligência artificial".
- A solução surgiu na década de 80 (MLP).

Vamos ao código!

PerceptronBacana.c

```c
#include<stdio.h>
#include<conio.h>
//Prof. Dr. Jefferson Beethoven Martins
int main()
{
    float entrada[2][4];
    int opc = 0;
    float limiar = 0;
    float w[4]; //Um peso para cada entrada.
    float target[2];//Duas possíveis saídas.
    float b;
    float alfa = 0.01;//taxa de aprendizagem 0<X<=1.
    float yLiq =0;
    int contCiclo=0;
    float y;
    float yTeste;
    int lin, col;
    int condErro=1;
    float teste;
    target[0] = -1;//Reconhecimento de A.
    target[1] = 1; //Reconhecimento de B.

    w[0] =  0.5;//O mais comum na literatura é [-0.5,+0.5].
```

Vamos ao código!

PerceptronBacana.c

```
23      w[0] =  0.5;//O mais comum na literatura é [-0.5,+0.5].
24      w[1] = -0.1;
25      w[2] =  0.4;
26      w[3] = -0.27;
27      b = 0.3256;
28
29      entrada[0][0]= -1;//Equivalente ao A (-1).
30      entrada[0][1]= -1;
31      entrada[0][2]=  1;
32      entrada[0][3]=  1;
33
34      entrada[1][0]=  1;//Equivalente ao B (1).
35      entrada[1][1]= -1;
36      entrada[1][2]=  1;
37      entrada[1][3]= -1;
38
39      while(opc != 3)
40      {
41          printf("\n\n ************ Programa Perceptron ************");
42          printf("\n\n Digite 1 para treinar a rede");
43          printf("\n Digite 2 para operar");
44          printf("\n Digite 3 para Sair\n ->");
45          scanf("%i",&opc);
```

Vamos ao código!

PerceptronBacana.c

```c
if(opc ==1)
{
    while(condErro == 1)
    {
        condErro = 0;
        lin = 0;
        while(lin<2)
        {
            yLiq = 0;
            col = 0;
            while(col<4)
            {
                yLiq = yLiq + (entrada[lin][col]*w[col]);
                col++;
            }
            yLiq = yLiq + b;
            if(yLiq >= limiar) //Função de ativação.
            {
                y = 1;
            }else{
                y= -1;
            }
            printf("\n y: %.2f - target: %.2f",y,target[lin]);
```

Vamos ao código!

PerceptronBacana.c

```c
            printf("\n y: %.2f - target: %.2f",y,target[lin]);
            //O segrego do perceptron está na linha abaixo.
            if(y != target[lin])
            {
                condErro = 1;
                col = 0;
                while(col<4)//Algoritmo para correção dos pesos e bias.
                {
                    w[col] = w[col] + (alfa * (target[lin] - y) * entrada[lin][col]);
                    col++;
                }
                b = b + alfa * (target[lin] - y);
            }
            lin++;
        }
        printf("\n Ciclo: %i \n",contCiclo);
        contCiclo++;
    }
    printf("\n Rede treinada!");
    col = 0;
    while(col<4)
    {
        printf("\n Peso[%i]: %.2f",col,w[col]);
```

Vamos ao código!

PerceptronBacana.c

```
90              printf("\n Peso[%i]: %.2f",col,w[col]);
91              col++;
92          }
93          printf("\n Bias: %.2f",b);
94      }
95      if(opc ==2)
96      {
97          printf("\n\t --->Testando a rede treinada");
98          printf("\n\n Teste com as entradas do treinamento");
99          //Mostrando as entradas.
100         lin = 0;
101         while(lin<2)
102         {
103             col = 0;
104             while(col<4)
105             {
106                 printf("\n Entrada[%i][%i]: %.2f",lin,col,entrada[lin][col]);
107                 col++;
108             }
109             lin++;
110             printf("\n -------------");
111         }
112         for(lin =0;lin<2;lin++)
```

Vamos ao código!

PerceptronBacana.c

```c
                }
                for(lin =0;lin<2;lin++)
                {
                    teste = 0;
                    for(col = 0; col < 4; col ++)
                    {//Somatório dos pesos.
                        teste= teste + (entrada[lin][col]*w[col]);
                    }
                    teste = teste + b;

                    if(teste >=limiar)
                    {
                        yTeste = 1;
                    }else{
                        yTeste = -1;
                    }
                    printf("\n Saida da rede[%i]: %.2f",lin,yTeste);
                }
            }
        }
    return(0);
}
```

Implemente Perceptron simples para este modelo abaixo (2 entradas – 2 padrões, saída -1 ou 1)

INSTITUTO FEDERAL
Triângulo Mineiro

[*] Untitled1

```
// Saída -1
entrada[0][0] = 1.0;
entrada[0][1] = 1.0;

// Saída 1
entrada[1][0] = 1.0;
entrada[1][1] = 1.00001;

target[0] = -1;
target[1] = 1;
```

Trocando Hebb por Perceptron (trabalho 02 – parte 01)

- Use o trabalho 01 para que testemos uma pequena mudança: Você receberá duas letras (A e B em matrizes 7x7) e, ao invés, de empregar Hebb, use Perceptron.

Trabalho 02 – parte 02: faça um programa (com interface gráfica) para testar cada porta lógica com Perceptron

INSTITUTO FEDERAL
Triângulo Mineiro

E

AND

A	B	S
0	0	0
0	1	0
1	0	0
1	1	1

NE

NAND

A	B	S
0	0	1
0	1	1
1	0	1
1	1	0

OU

OR

A	B	S
0	0	0
0	1	1
1	0	1
1	1	1

NOU

NOR

A	B	S
0	0	1
0	1	0
1	0	0
1	1	0

OU Exclusivo

A	B	S
0	0	0
0	1	1
1	0	1
1	1	0

Trabalho 02 – parte 02

- O desafio é: crie um programa que permita que o usuário escolha qual porta lógica deseja treinar uma rede perceptron. O programa deve mostrar os valores finais dos pesos e bias, após o treinamento. Fique atento à porta OU Exclusivo.
- Não se esqueçam, todos os projetos devem ser apresentados em sala.

Referências e agradecimentos

- HAYKIN, Simon. Redes neurais: princípios e prática. Bookman Editora, 2007.
- FAUSETT, Laurene. Fundamentals of neural networks: architectures, algorithms, and applications. Prentice-Hall, Inc., 1994.
- Silva, I. N., Spatti, D. H., & Flauzino, R. A. (2010). Redes neurais artificiais para engenharia e ciências aplicadas curso prático. Artliber.

- Agradecimentos especiais ao professor José Ricardo Gonçalves Manzan.